EXPERIMENT NO 8

Name:- Amogh Prafulla Bapat

PRN:-B25ET1201 Div:- SY3 Batch:- C3

INPUT CODE

```
#include <stdio.h>
#include <stdlib.h>
struct Node {
    int data;
    struct Node* left;
    struct Node* right;
};
struct Node* createNode(int value) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    if (newNode == NULL) {
        printf("Memory allocation failed!\n");
        return NULL;
    }
    newNode->data = value;
    newNode->left = NULL;
    newNode->right = NULL;
    return newNode;
}
struct Node* insert(struct Node* root, int value) {
    if (root == NULL) {
        return createNode(value);
    }
    if (value < root->data) {
        root->left = insert(root->left, value);
    }
    else if (value > root->data) {
        root->right = insert(root->right, value);
    }
    return root;
}
struct Node* search(struct Node* root, int value) {

    if (root == NULL || root->data == value) {
        return root;
    }
```

```
    if (value < root->data) {
        return search(root->left, value);
    }
    else {
        return search(root->right, value);
    }
}
void inorderTraversal(struct Node* root) {
    if (root != NULL) {
        inorderTraversal(root->left);
        printf("%d ", root->data);
        inorderTraversal(root->right);
    }
}
void preorderTraversal(struct Node* root) {
    if (root != NULL) {
        printf("%d ", root->data);
        preorderTraversal(root->left);
        preorderTraversal(root->right);
    }
}
void postorderTraversal(struct Node* root) {
    if (root != NULL) {
        postorderTraversal(root->left);
        postorderTraversal(root->right);
        printf("%d ", root->data);
    }
}
int main() {
    struct Node* root = NULL;
    struct Node* result;
    int value;
    root = insert(root, 50);
    insert(root, 30);
    insert(root, 20);
    insert(root, 40);
    insert(root, 70);
    insert(root, 60);
    insert(root, 80);
    printf("Inorder traversal (sorted): ");
    inorderTraversal(root);
    printf("\n");
```

```
    printf("Preorder traversal:         ");
    preorderTraversal(root);
    printf("\n");
    printf("Postorder traversal:        ");
    postorderTraversal(root);
    printf("\n");
    printf("\nEnter value to search: ");
    scanf("%d", &value);
    result = search(root, value);
    if (result != NULL) {
        printf("%d is found in the BST.\n", value);
    }
    else {
        printf("%d is not found in the BST.\n", value);
    }
    return 0;
}
```

OUTPUT



